

For our project we plan to use the Modified National Institute of Standards and Technology (MNIST) handwritten database to train a Convolutional Neural Network (CNN) to recognize the basic digits 0-9. To train the CNN we plan to use PyTorch, a framework for efficiently implementing and training our model on a GPU. PyTorch is specifically appealing for our project as it is well documented and relatively easy to use, as well as allowing for very high efficiency matrix operation, reducing the processing power required to train the model. For our GPU we plan to use a Nvidia 5050 Laptop GPU with 2560 CUDA cores, which should be more than enough to train our model.

The design of our model will include two convolution layers and two pooling layers, which use the 28x28 pixel MNIST images as inputs. The convolution layers will allow the model to detect patterns and represent the images in a more complex fashion. The pooling layers reduce the size of the subsequent layers and reduce the computational costs. We intend to use average pooling to reduce the size of the subsequent layers as average pooling retains the most general information from an area rather than only focusing on the most prominent features. For something as visually diverse as handwritten numbers, this is important because while neatly handwritten numbers are visually distinct, the messier and blurrier the handwriting is, the harder it is to distinguish between numbers, such as the difference between a messy looking 1 and 7. By choosing average pooling over max pooling, we can focus on the overall information better, rather than cherry-pick the most intense features-possibly leading to wrong conclusions due to the visual diversity of handwriting. Pooling will also reduce the risk of the model overfitting and “memorizing” the training data. The model will also have two fully connected layers to map our features to the final outputs. There will then be a final output layer corresponding to the ten handwritten digits 0-9.

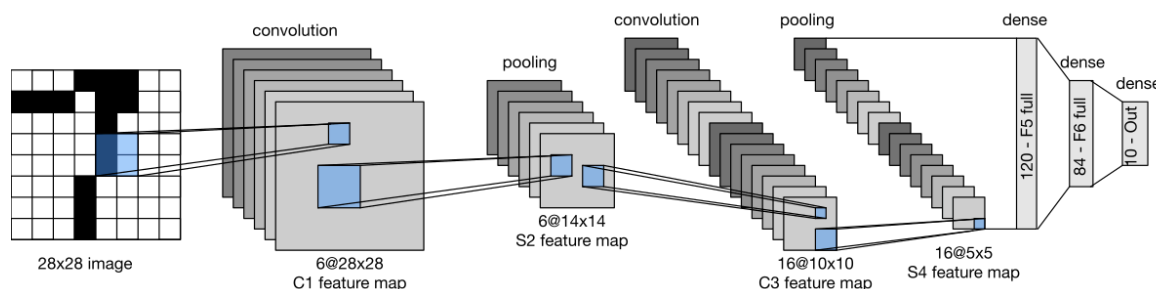


Figure 1. CNN architecture

Our model will be trained in FP32, as the dataset and model will be small enough and our hardware fast enough to not require training optimizations like PyTorch’s automatic mixed precision (AMP). Furthermore, FP32 ensures we have sufficient resolution when training to save the weights required to capture the small gradients in the MNIST dataset. However, for inference we want to quantize our model to reduce the computational cost of using the model. We will quantize the model from FP32 to a four bit integer representation after training. We intend to use a linear, symmetric power of two quantization scheme to take advantage of bitshifting on an FPGA. With a zero point of 0, this allows us to not have to use more adders to quantize/de-quantize. Since we are scaling by factors of 2 to map our FP32 weights to 4 bit integers, all this requires is bit shifting left or right, massively reducing the cost compared to using a multiplier to quantize. Furthermore, we will be using a saturated model to maintain high

accuracy, with the threshold to be determined later through research and careful design. To further increase the efficiency of our model we will design several systolic arrays that will arrange Processing Elements (PEs) in a fashion that will reduce expensive writes and reads to DRAM and use nearby data which can be drawn from neighboring PEs and buffers. We plan to implement four systolic arrays: Weight Stationary, Input Stationary, Output Stationary, and Row Stationary.

We will implement these arrays on an Intel DE1-SoC FPGA, and compare the performance differences for each. We plan to record the computation time, FPGA area, and memory use of each to determine which is the most efficient method for representing the model.